```
  def hi(name \\ "Reader") do
    "Hi #{name}"
  end
end
```

This code also demonstrates that Elixir supports default function arguments using the \\ operator. This creates two versions of our function—hi/0 and hi/1—that have different values for the name argument.

When you run this in IEx, you'll see that our functions behave differently based on the number of arguments and the data in those arguments:

```
$ iex -S mix

iex> Examples.Modules.Two.hello()
"Hello"

iex> Examples.Modules.Two.hello("Steve")
"Hello Steve"

iex> Examples.Modules.Two.hello("Mike")
"Hello Mike, Hello Joe"

iex> Examples.Modules.Two.hi()
"Hi Reader"

iex> Examples.Modules.Two.hi("Steve")
"Hi Steve"
```

This technique may be completely new to you because Ruby doesn't have method overloading. If you define a Ruby method with the same name as a previously defined method, then the old definition will be skipped. For example, the following Ruby method returns "hello":

ruby_examples/two.rb

```
class Overload
  def hello(name)
    "Hi #{name}"
  end

  def hello
    "hello"
  end
end

Overload.new.hello #> "hello"
Overload.new.hello("Steve") #> ArgumentError (given 1, expected 0)
```

Even if the Ruby method was defined with a different number of arguments, there would still be only one version of that method. You would need to implement a different behavior in the body of the method instead of having multiple definitions.